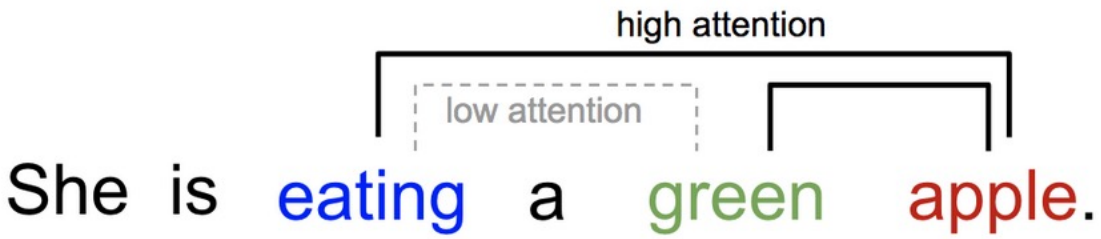
NLP 243 Deep Learning for Natural Language Processing

Fall 2024

Including slides adapted from Dilek-Hakkani-Tur, Vivian Chen and depictions from Chris Olah's blogpost

Attention mechanism

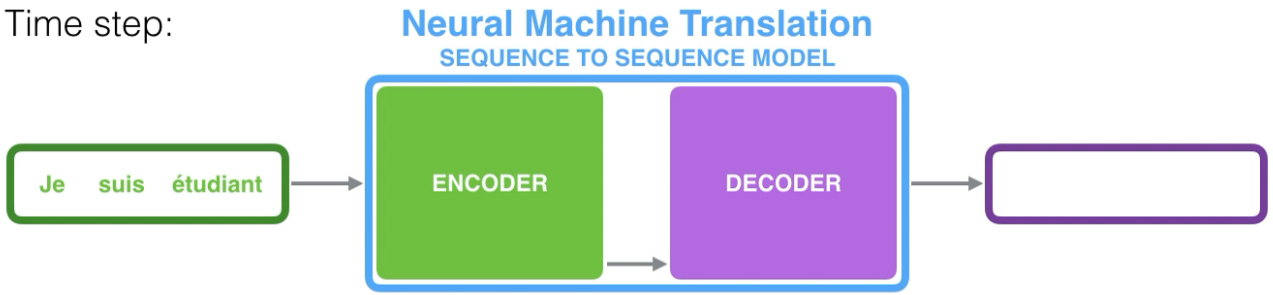


Attention: vector of importance weights:

Paper: Effective Approaches to Attention-based Neural Machine Translation

Seq2seq Models With Attention

Each timestamp for the encoder or decoder is an RNN processing its inputs and generating an output for that time step.
Since the encoder and decoder are both RNNs, each time step one of the RNNs does some processing, it updates its hidden state based on its inputs and previous inputs it has seen.

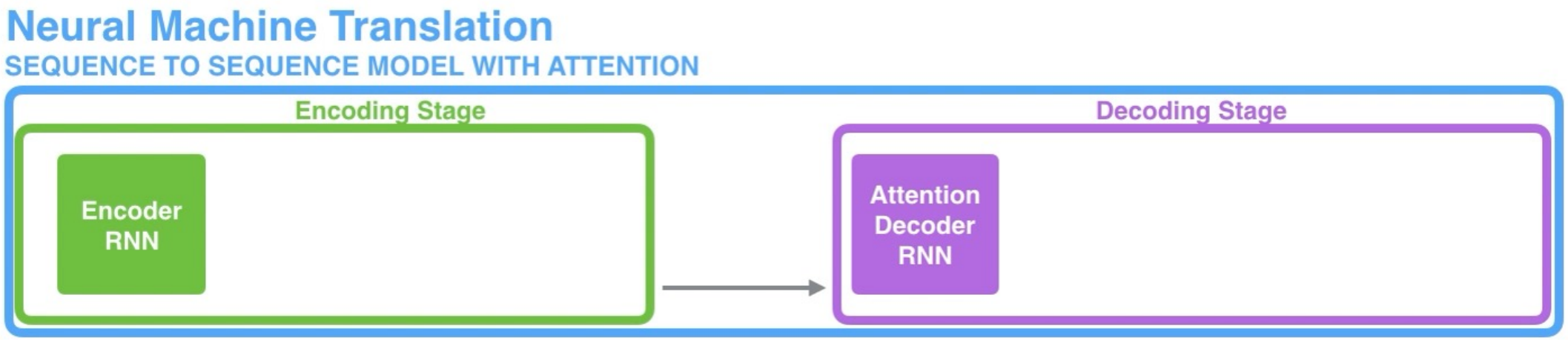


The context vector turned out to be a bottleneck for these types of models.
It made it challenging for the models to deal with long sentences.
A solution was proposed in Bahdanau et al., 2014 and Luong et al., 2015.
These papers introduced and refined a technique called “Attention”, which highly improved the quality of machine translation
Attention allows the model to focus on the relevant parts of the input sequence as needed.

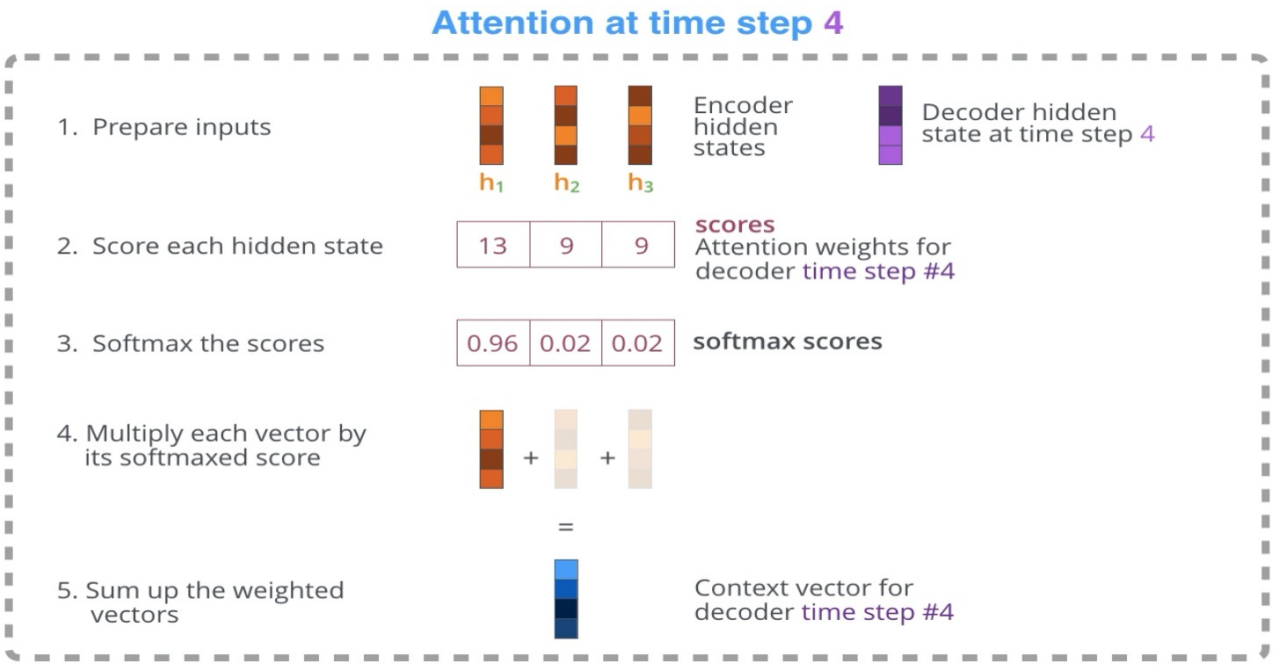
Seq2seq Models With Attention

An attention model differs from a classic sequence-to-sequence model in two main ways:
First, the encoder passes a lot more data to the decoder. Instead of passing the last hidden state of the encoding stage, the encoder passes *all* the hidden states to the decoder:

Second, in order to focus on the parts of the input that are relevant to this decoding time step, the decoder does the following:
1.Look at set of encoder hidden states it received – each encoder hidden state is most associated with a certain word in the input sentence
2.Give each hidden state a score (let's ignore how the scoring is done for now)
3.Multiply each hidden state by its softmaxed score, thus amplifying hidden states with high scores, and drowning out hidden states with low scores



Seq2seq Models With Attention



1.Each encoder hidden state is most associated with a certain word in the input sentence

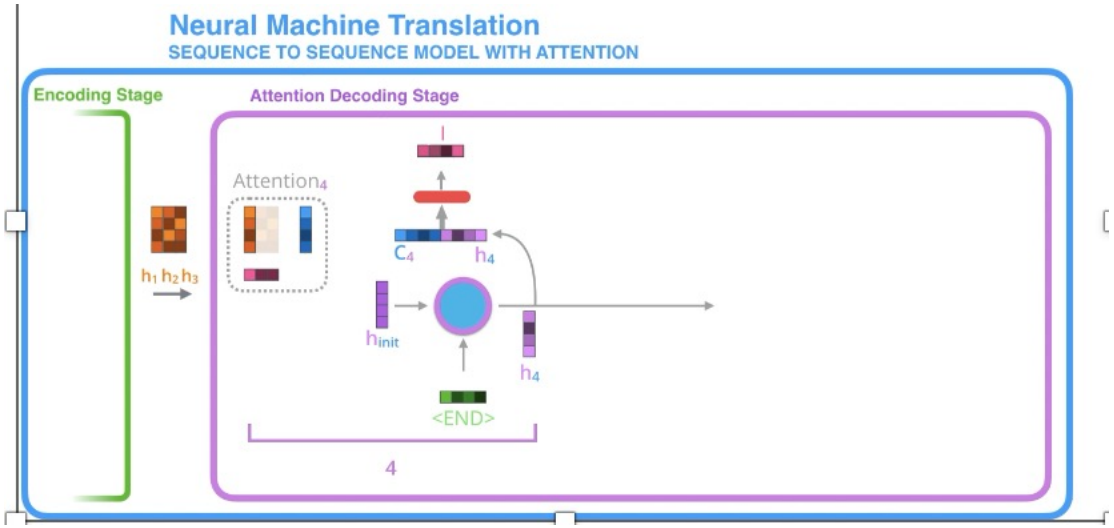
2.Give each hidden state a score

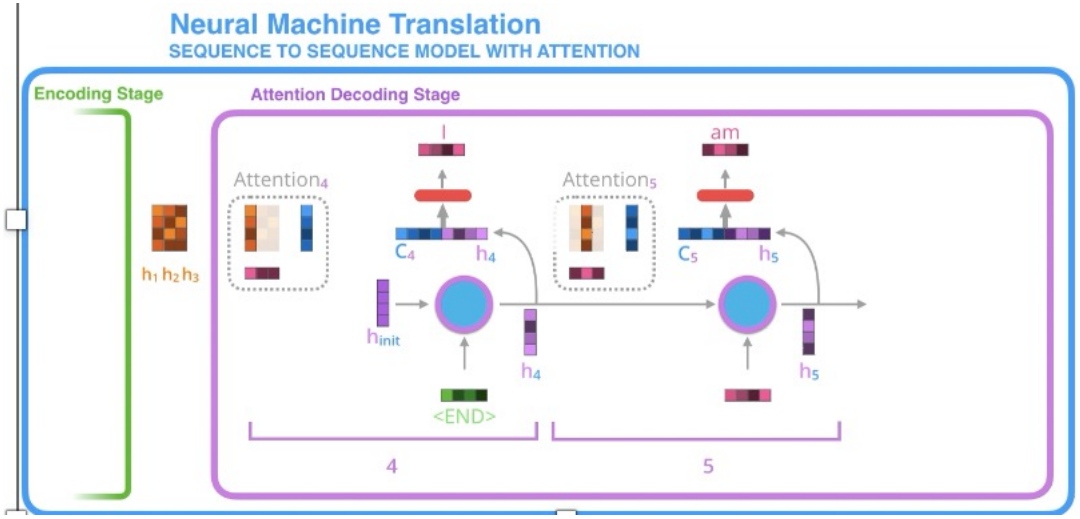
3.Multiply each hidden state by its softmaxed score, thus amplifying hidden states with high scores, and drowning out hidden states with low scores

This scoring exercise is done at each time step on the decoder sid

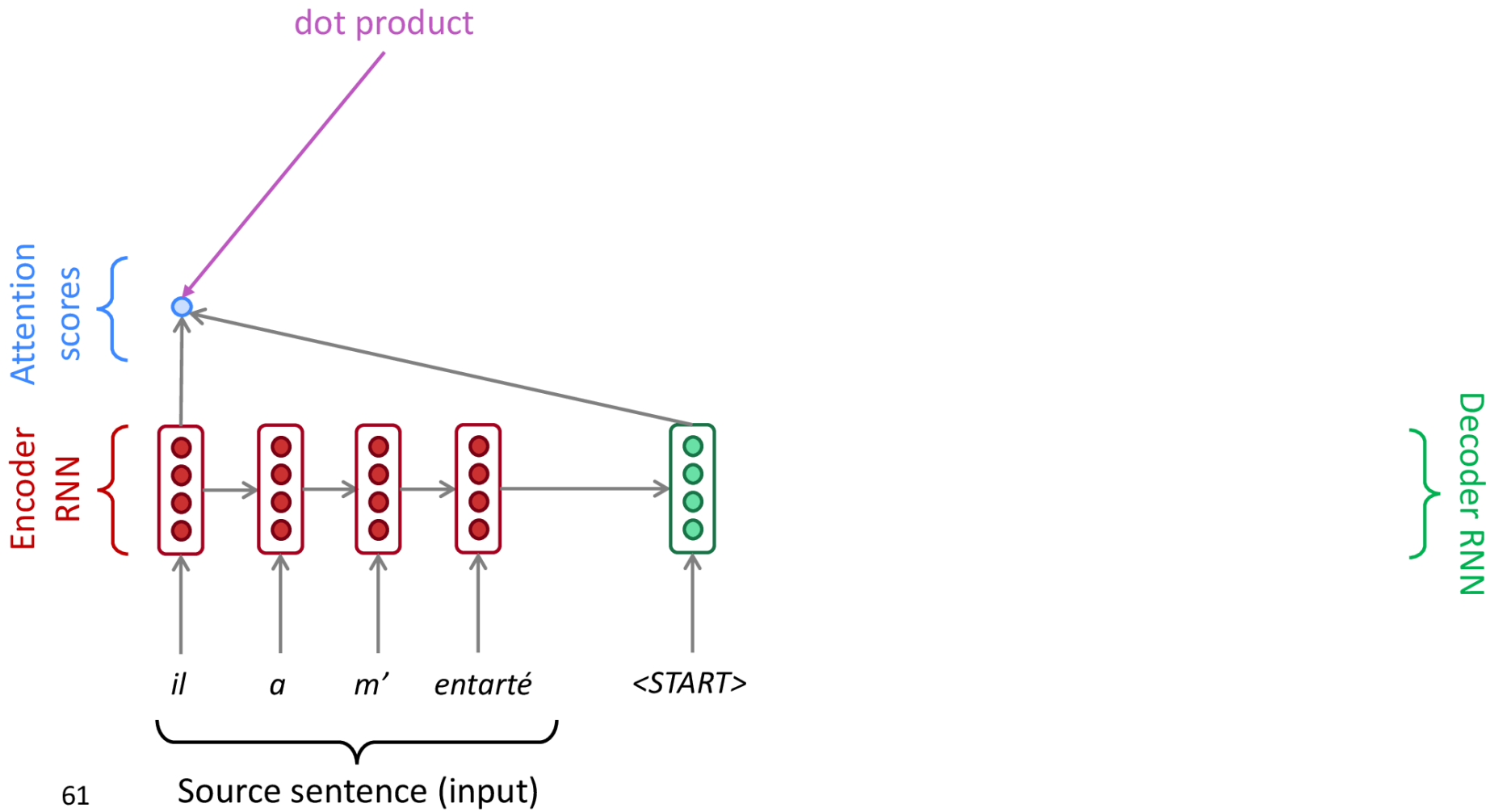
Seq2seq Models With Attention

1.The attention decoder RNN takes in the embedding of the <END> token, and an initial decoder hidden state.
2.The RNN processes its inputs, producing an output and a new hidden state vector (h4). The output is discarded.
3.Attention Step: We use the encoder hidden states and the h4 vector to calculate a context vector (C4) for this time step.
4.We concatenate h4 and C4 into one vector.
5.We pass this vector through a feedforward neural network (one trained jointly with the model).
6.The output of the feedforward neural networks indicates the output word of this time step.
7.Repeat for the next time steps

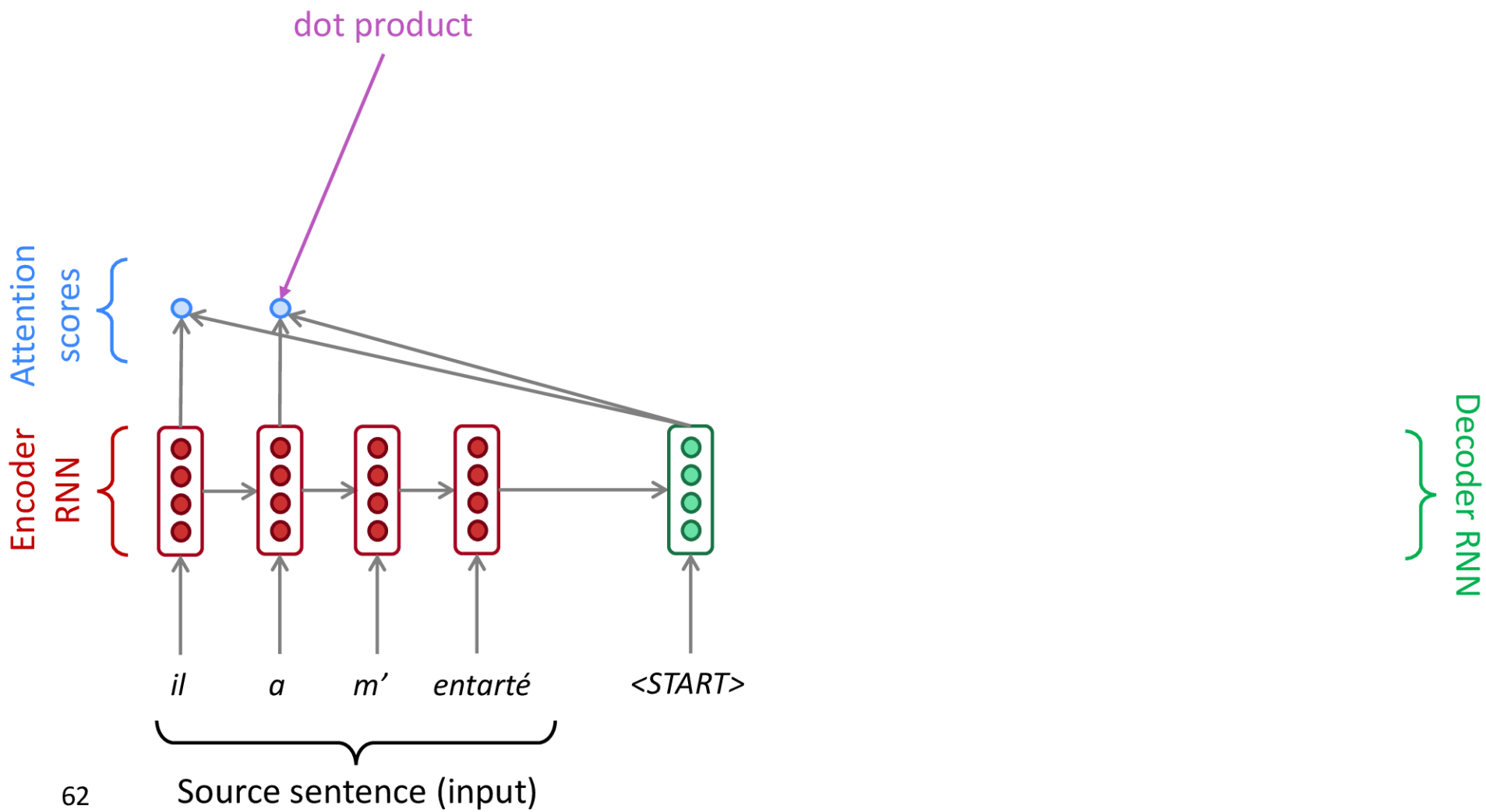




Sequence-to-sequence with attention

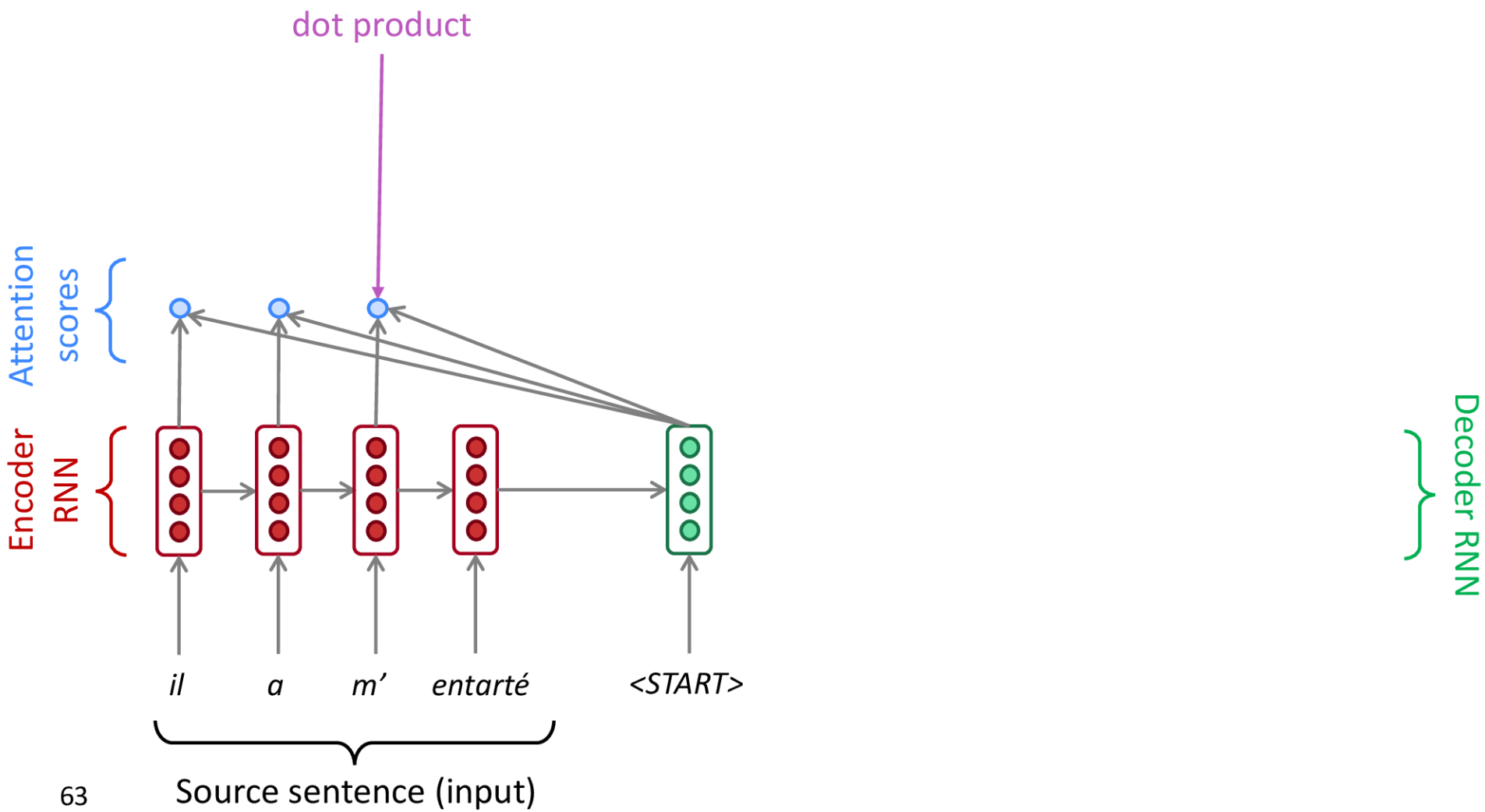


Sequence-to-sequence with attention



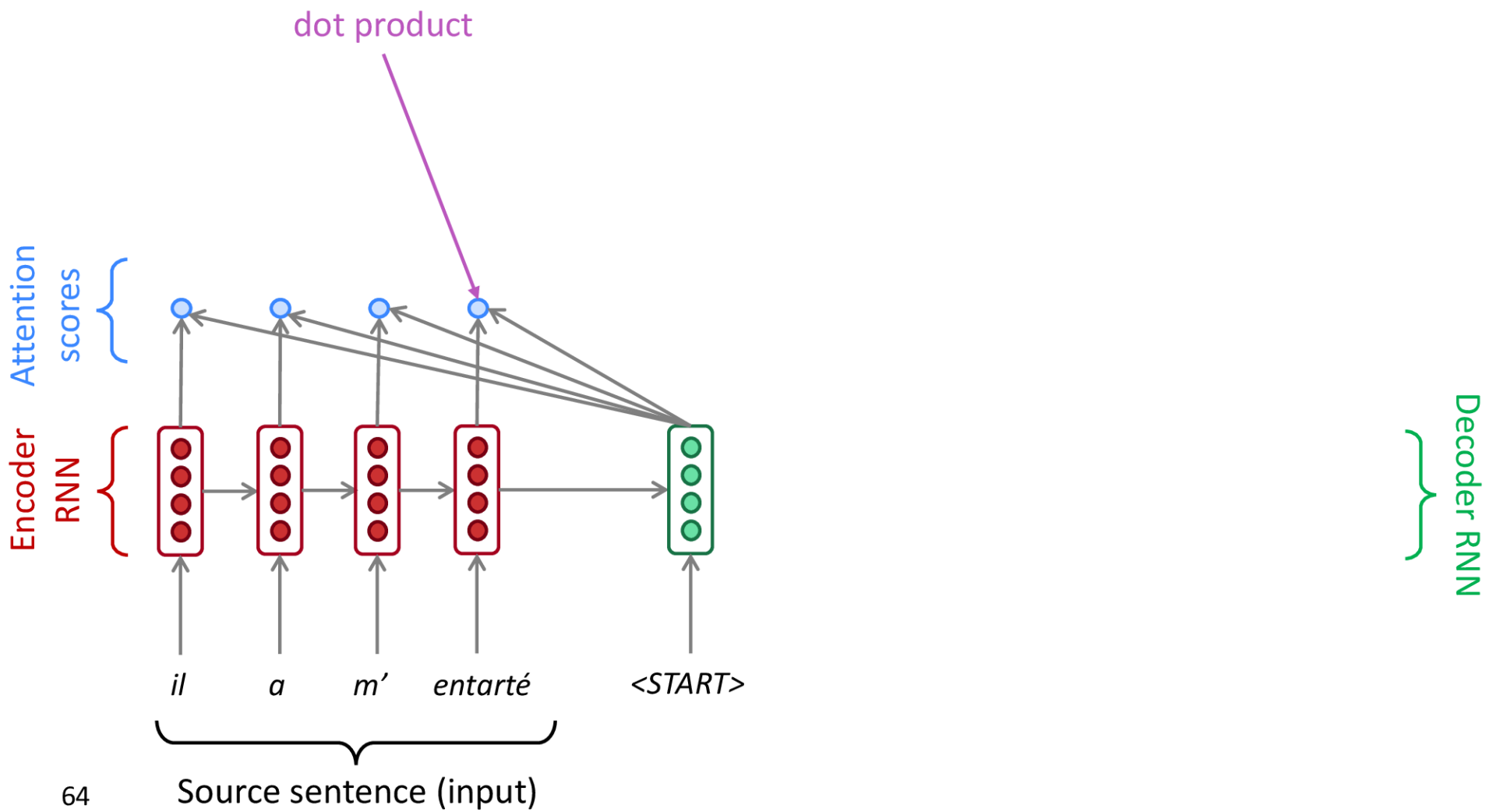
Source sentence (input)

Sequence-to-sequence with attention



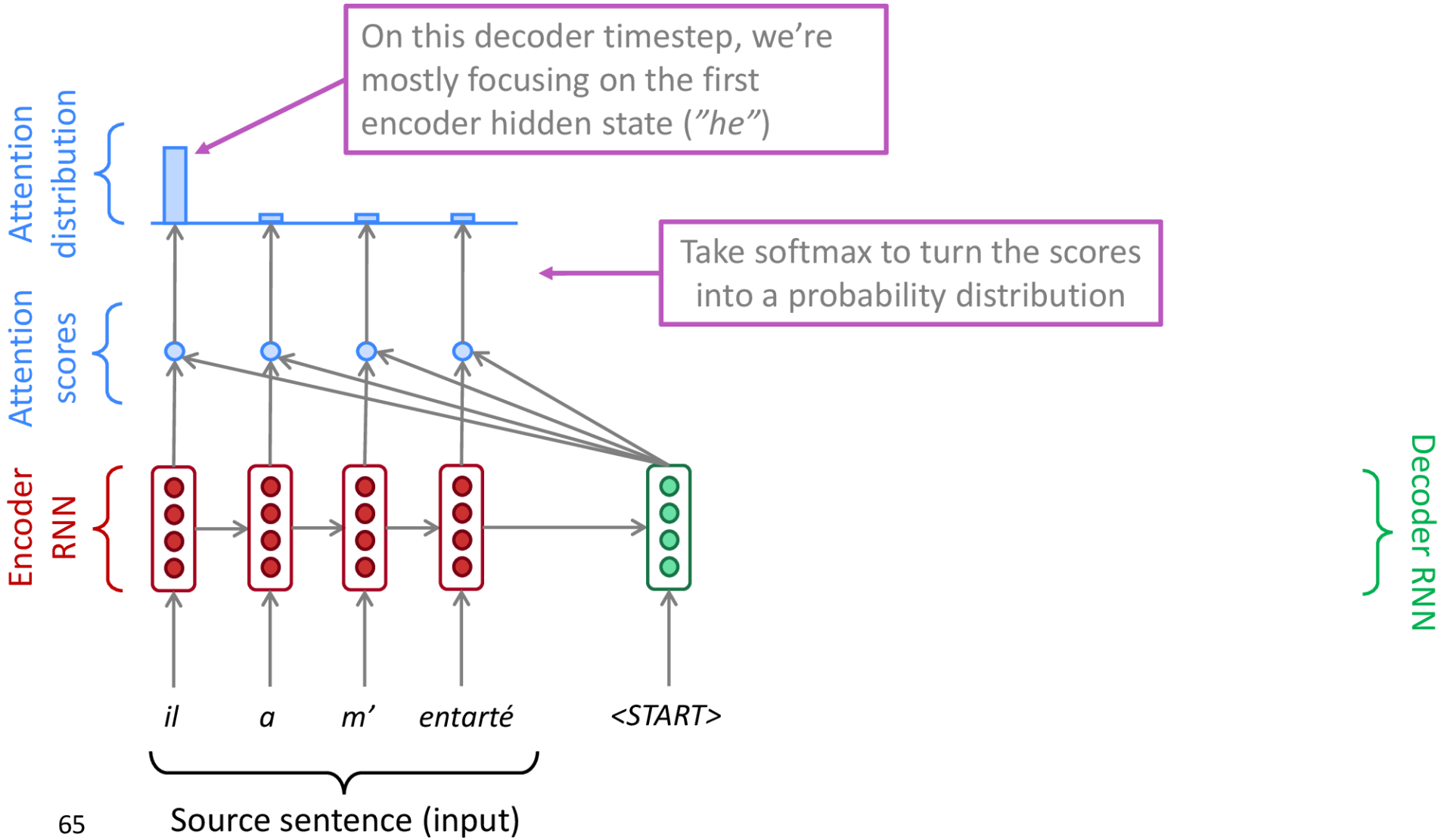
Source sentence (input)

Sequence-to-sequence with attention

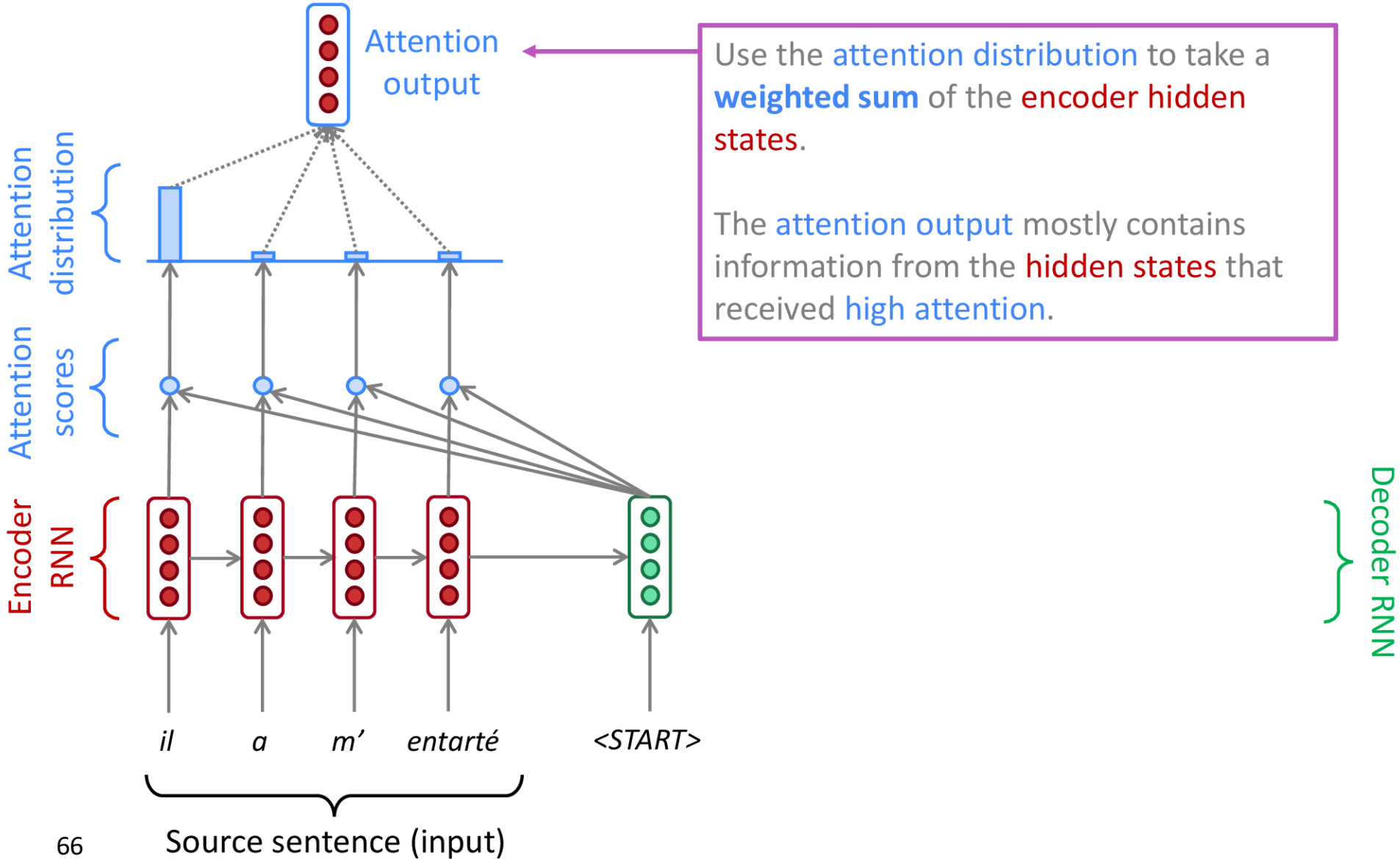


Source sentence (input)

Sequence-to-sequence with attention



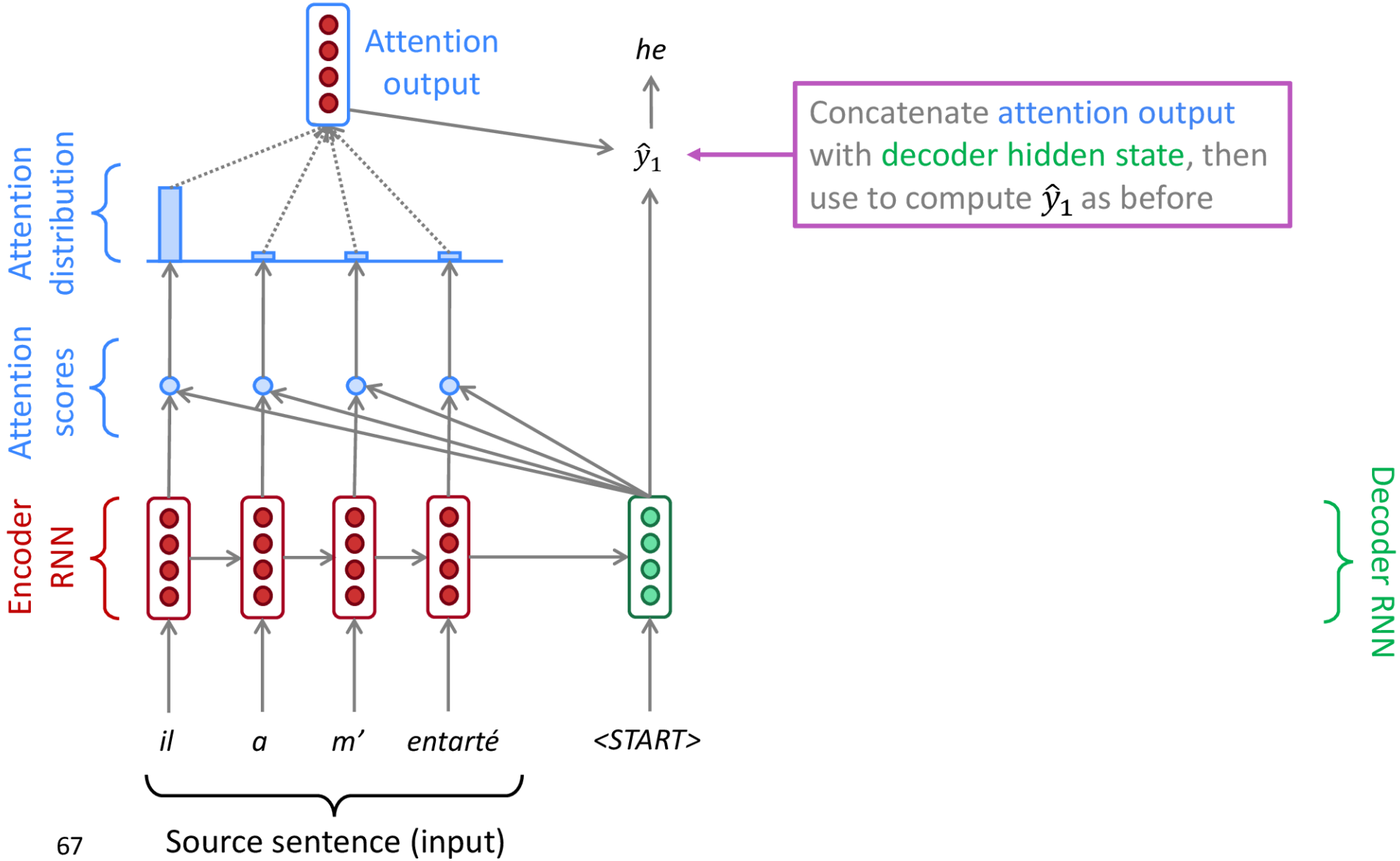
Sequence-to-sequence with attention



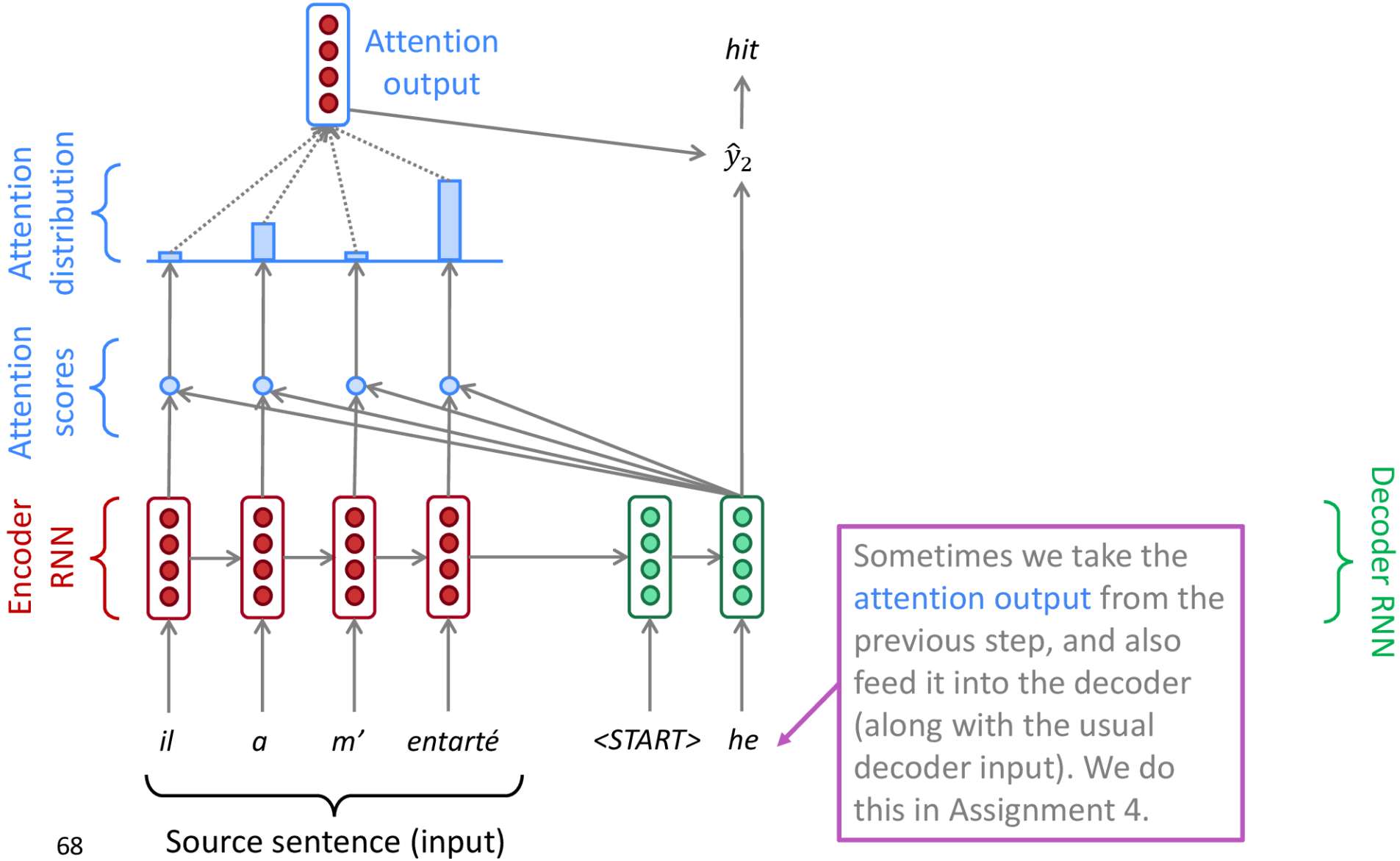
Use the attention distribution to take a **weighted sum** of the encoder hidden states.

The attention output mostly contains information from the hidden states that received high attention.

Sequence-to-sequence with attention

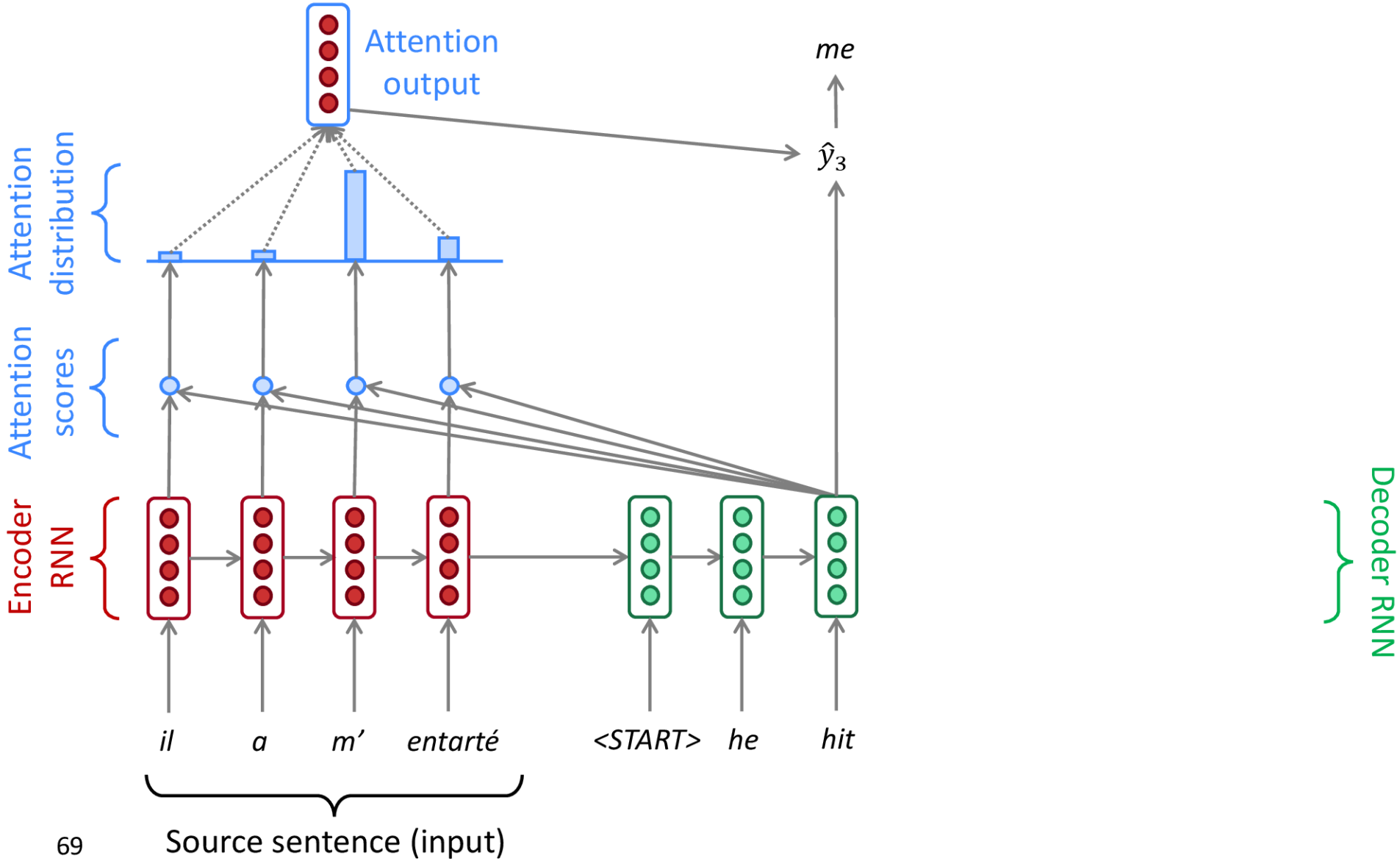


Sequence-to-sequence with attention

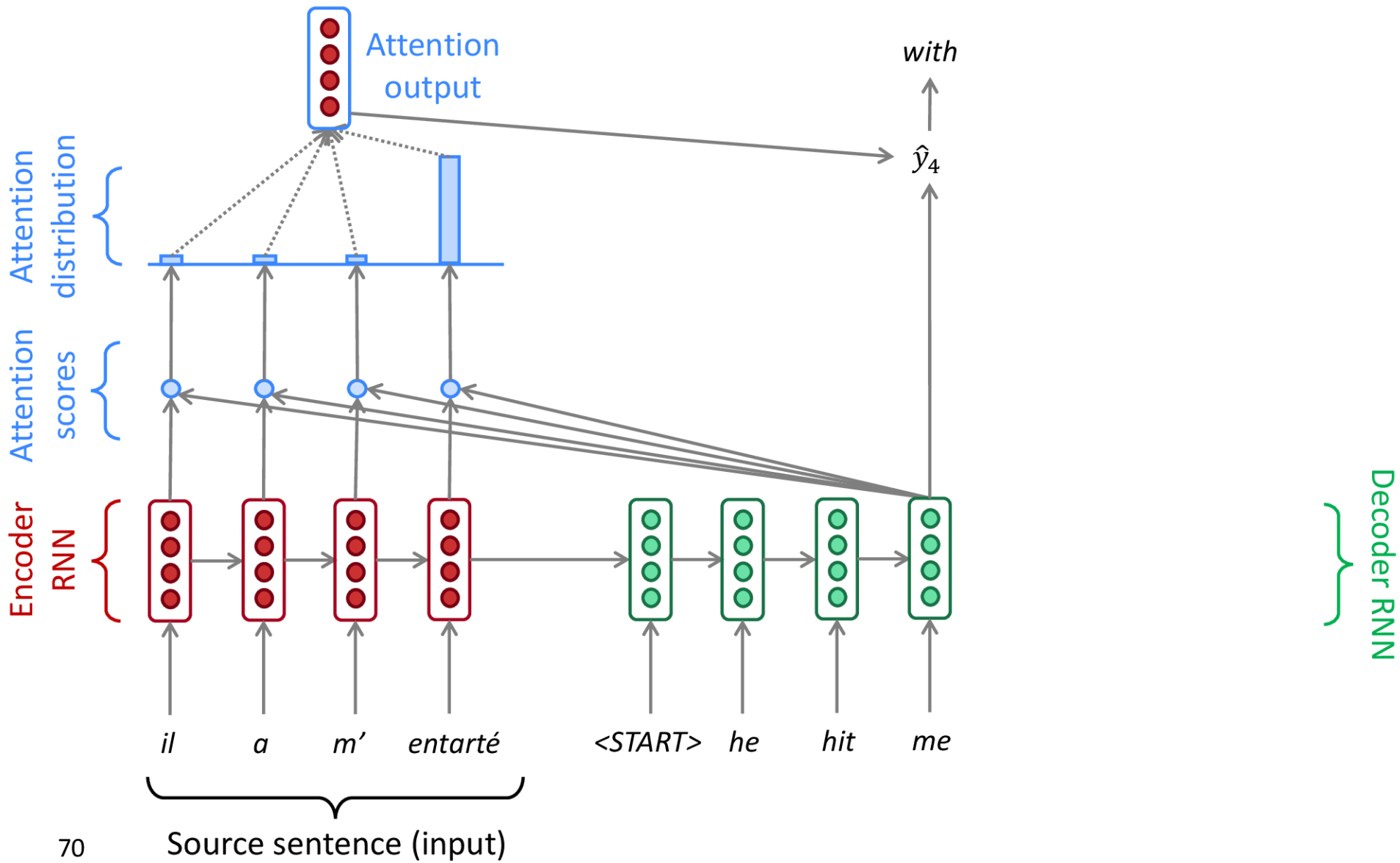


Sometimes we take the attention output from the previous step, and also feed it into the decoder (along with the usual decoder input). We do this in Assignment 4.

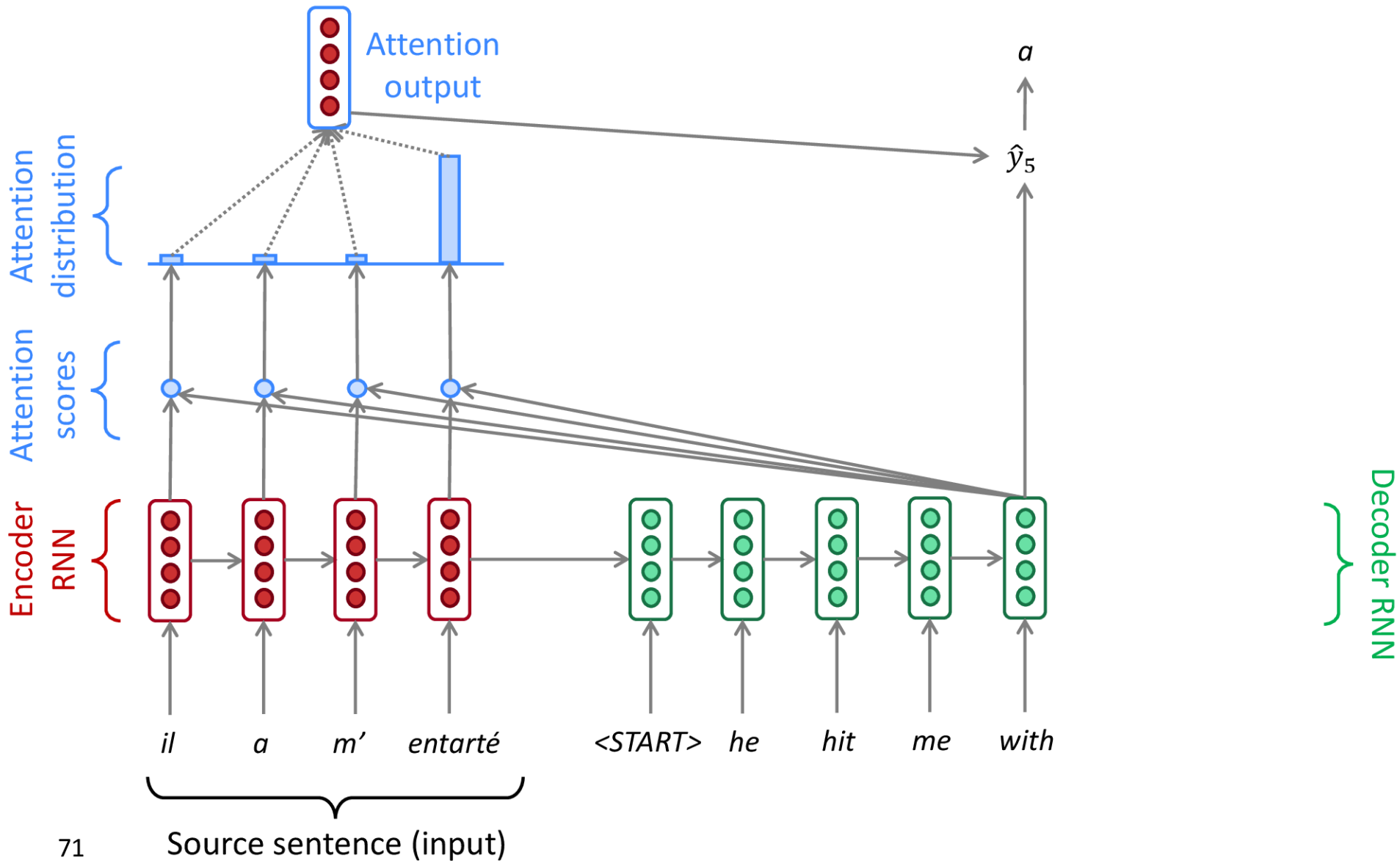
Sequence-to-sequence with attention



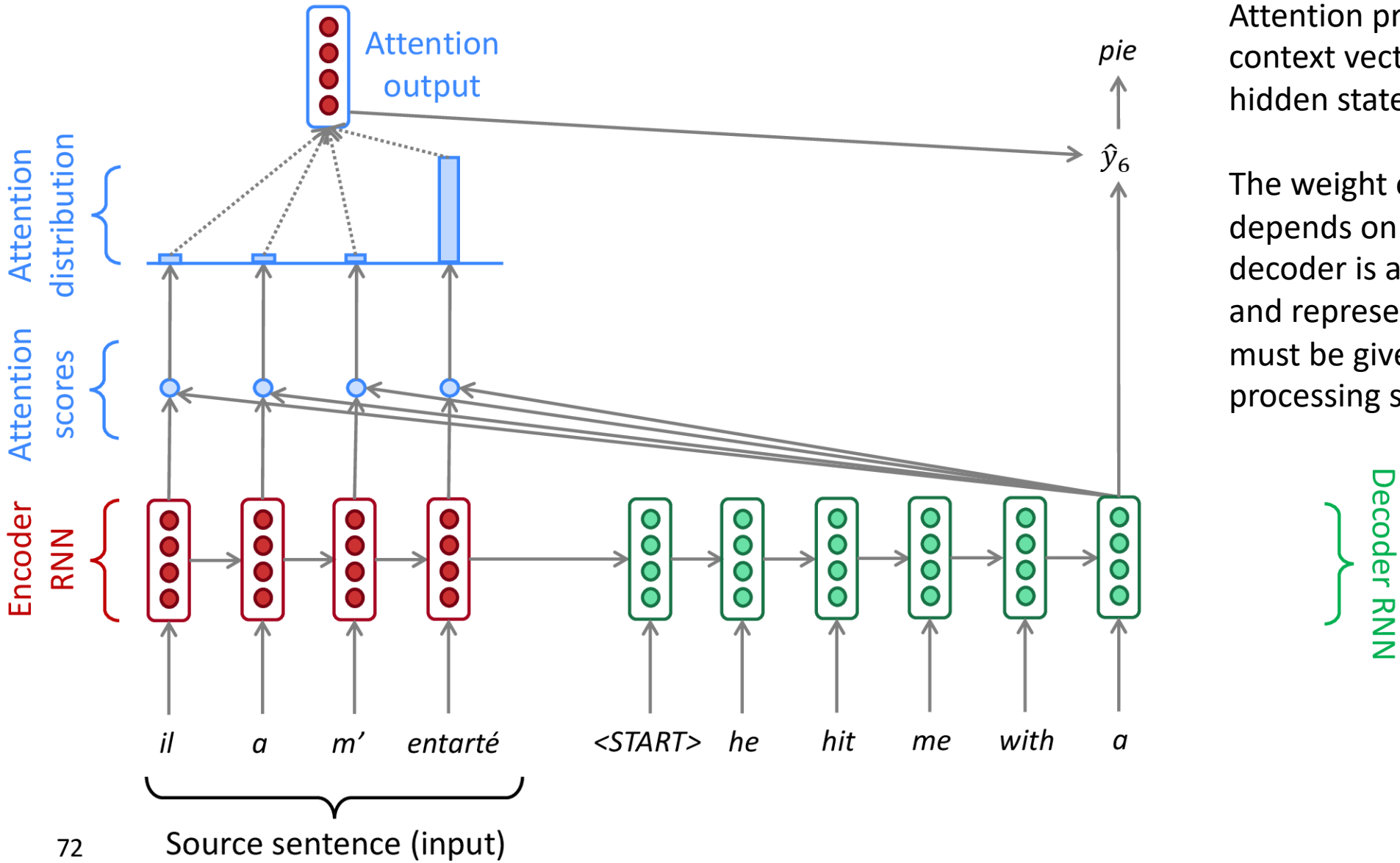
Sequence-to-sequence with attention



Sequence-to-sequence with attention

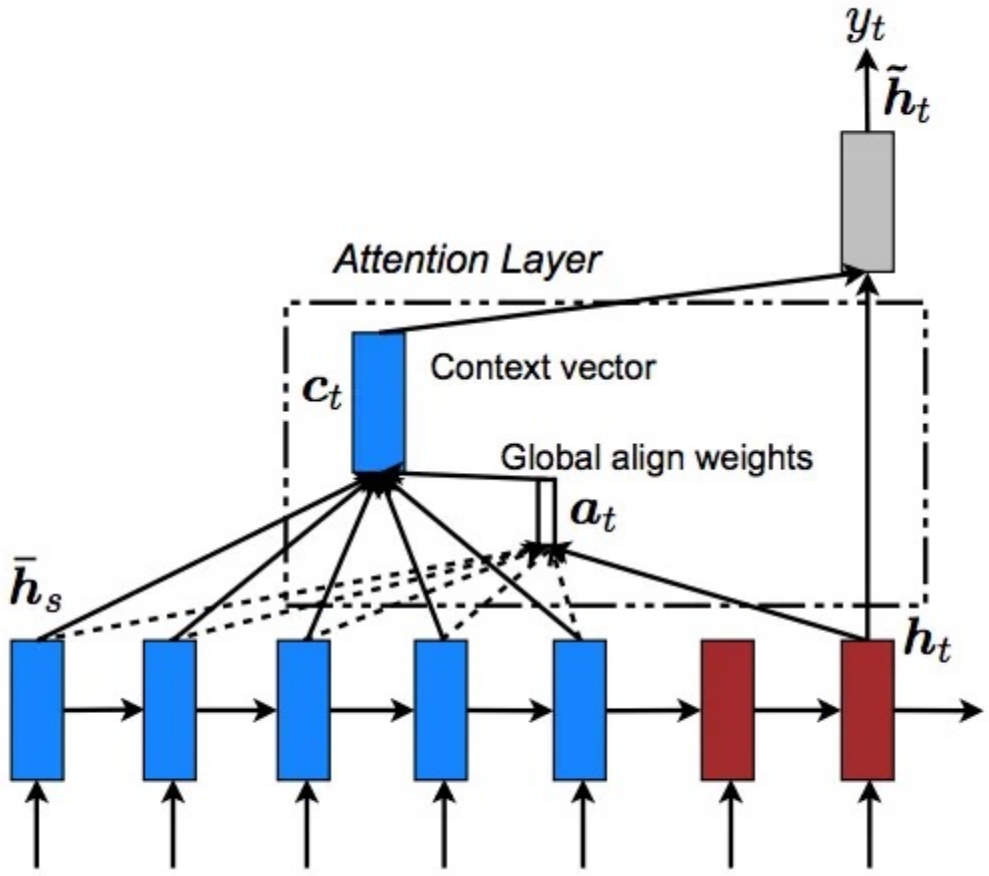


Sequence-to-sequence with attention



Attention produces a single fixed-size context vector from all the encoder hidden states (with a weighted sum).

The weight of each hidden state depends on the input word that the decoder is accepting at the moment, and represents the "attention" that must be given to that context when processing such input word.



h(t): Hidden target state
c(t): Source side context vector
y(t): Current target word
h_bar(t): Attentional hidden state
a(t): Alignment vector

1. First take as input the hidden state h(t) at the top layer of a stacking LSTM. (The brown cell / The target state of the decoder)
2. Derive c(t) to capture relevant source side information to help predict y(t) (Top)blue cell).
c(t) is basically the context that you have built for every word depending upon its alignment weights and hidden state of encoders.
3. Compute h_bar(t) from a simple concatenation of h(t) and c(t) (Top grey cell).

$$p(y_t|y_{<t}, x) = \text{softmax}(\boldsymbol{W_s}\tilde{\boldsymbol{h}}_t)$$

4. Attentional vector is transformed using the softmax layer to produce the predictive distribution.

$$p(y_t|y_{<t}, x) = \text{softmax}(\boldsymbol{W_s}\tilde{\boldsymbol{h}}_t)$$

Global Attention

Alignment Vector (a(t))

$$\boldsymbol{a}_t(s) = \text{align}(\boldsymbol{h}_t, \bar{\boldsymbol{h}}_s) = \frac{\exp\left(\text{score}(\boldsymbol{h}_t, \bar{\boldsymbol{h}}_s)\right)}{\sum_{s'} \exp\left(\text{score}(\boldsymbol{h}_t, \bar{\boldsymbol{h}}_{s'})\right)}$$

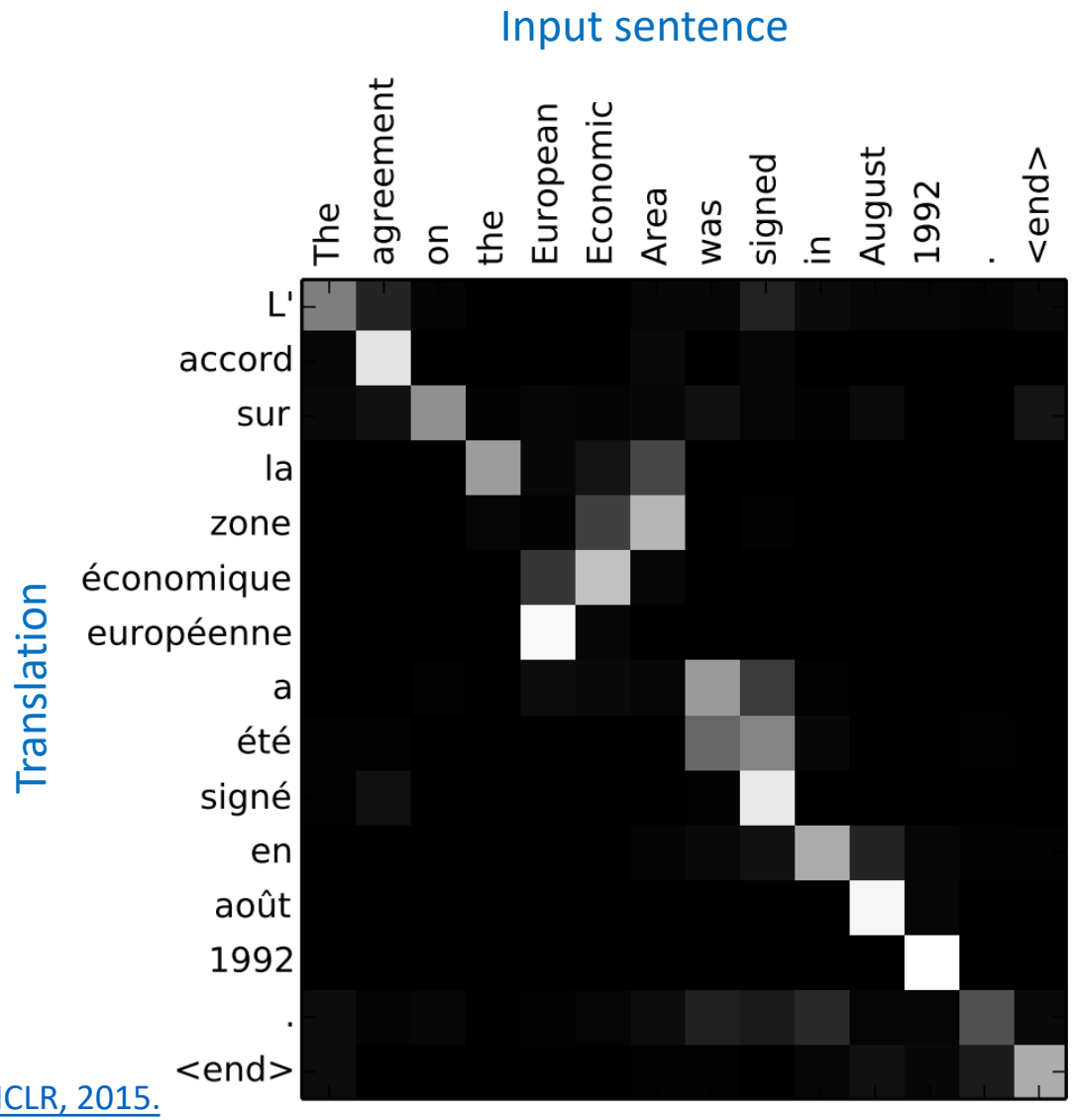
The score is a content-based function for which any of the following alternatives could have been used:

$$\text{score}(\boldsymbol{h}_t, \bar{\boldsymbol{h}}_s) = \begin{cases} \boldsymbol{h}_t^\top \bar{\boldsymbol{h}}_s & \textit{dot} \\ \boldsymbol{h}_t^\top \boldsymbol{W}_a \bar{\boldsymbol{h}}_s & \textit{general} \\ \boldsymbol{v}_a^\top \tanh\left(\boldsymbol{W}_a [\boldsymbol{h}_t; \bar{\boldsymbol{h}}_s]\right) & \textit{concat} \end{cases}$$

In Bahdanau's paper:
The alignment score is parametrized by a **feed-forward network** with a single hidden layer
Network is jointly trained with other parts of the model

Visualization of Attention for MT

- Attention probability matrix can be useful for visualization.
- Pixels show the weight α_{ij} of the annotation of the jth source word for the ith target word
- Plot is grayscale (0: black, 1: white)



Bahdanu et al., Neural Machine Translation by Jointly Learning to Align and Translate. ICLR, 2015.

Seq2seq with Local Attention

When Global Attention is applied, a lot of computation occurs.
All the hidden states must be taken into consideration, concatenated into a matrix, and processed by a NN to compute their weights.

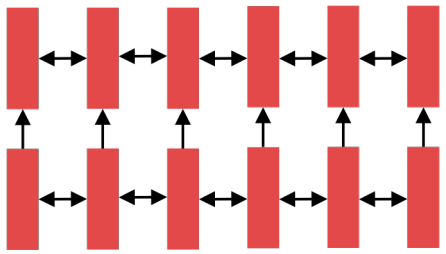
Can we reduce the number of computations, without sacrificing quality?
Yes, with Local Attention!

Local Attention is an Attention mechanism that considers only a subset of all the hidden states in creating the context vector.

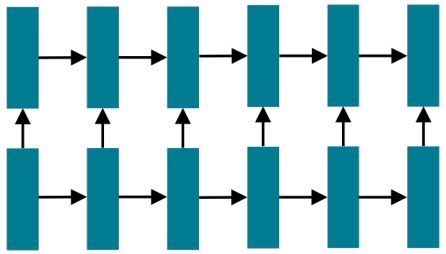
Monotonic Alignment Local Attention
Selects the subset of hidden states by keeping only the hidden states closer to the current translation step.

Recurrent models for (most) NLP!

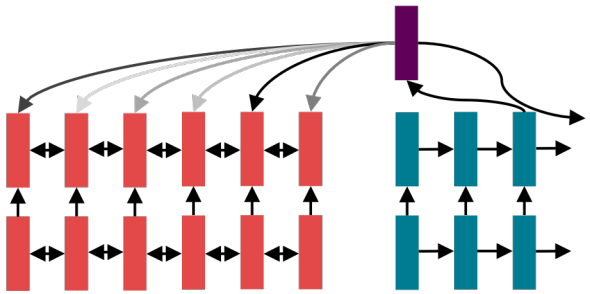
- Circa 2016, the de facto strategy in NLP is to **encode** sentences with a bidirectional LSTM: (for example, the source sentence in a translation)



- Define your output (parse, sentence, summary) as a sequence, and use an LSTM to generate it.

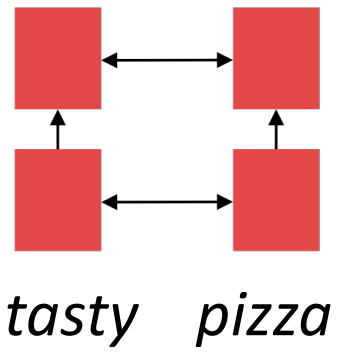


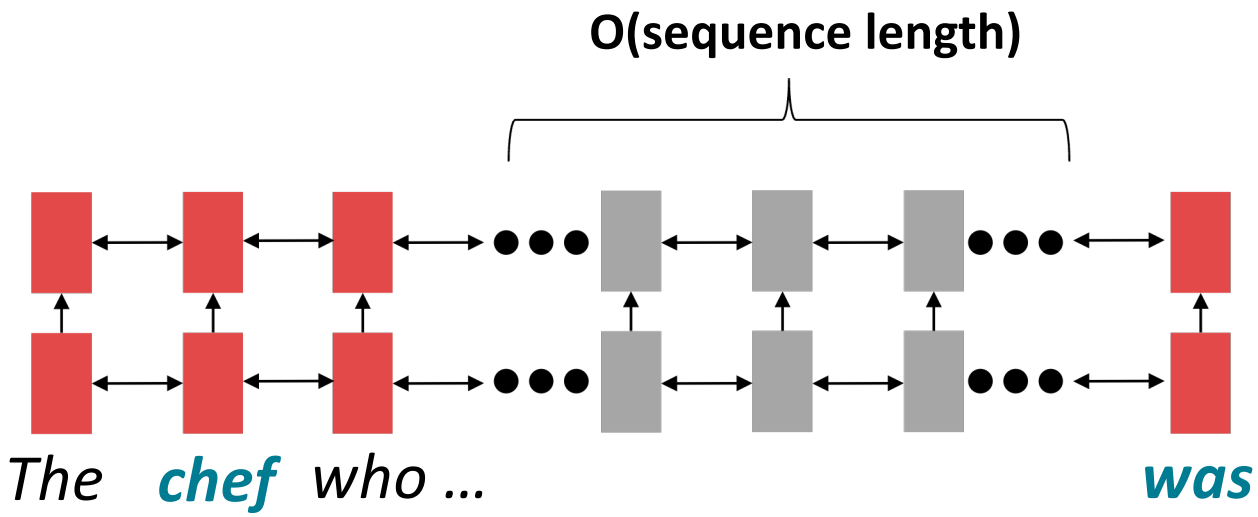
- Use attention to allow flexible access to memory



Issues with recurrent models: Linear interaction distance

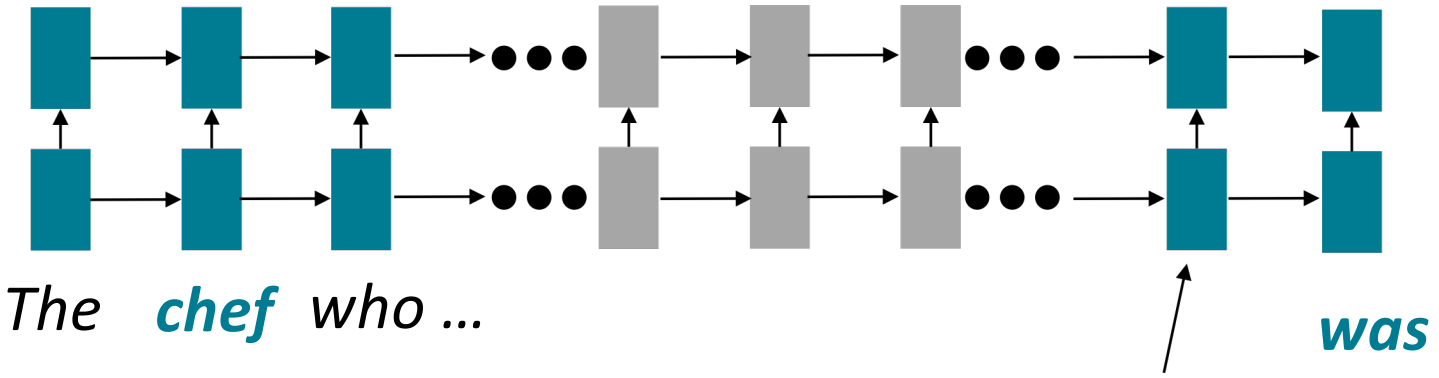
- RNNs are unrolled "left-to-right".
- This encodes linear locality: a useful heuristic!
  - Nearby words often affect each other's meanings
- **Problem:** RNNs take **O(sequence length)** steps for distant word pairs to interact.





Issues with recurrent models: Linear interaction distance

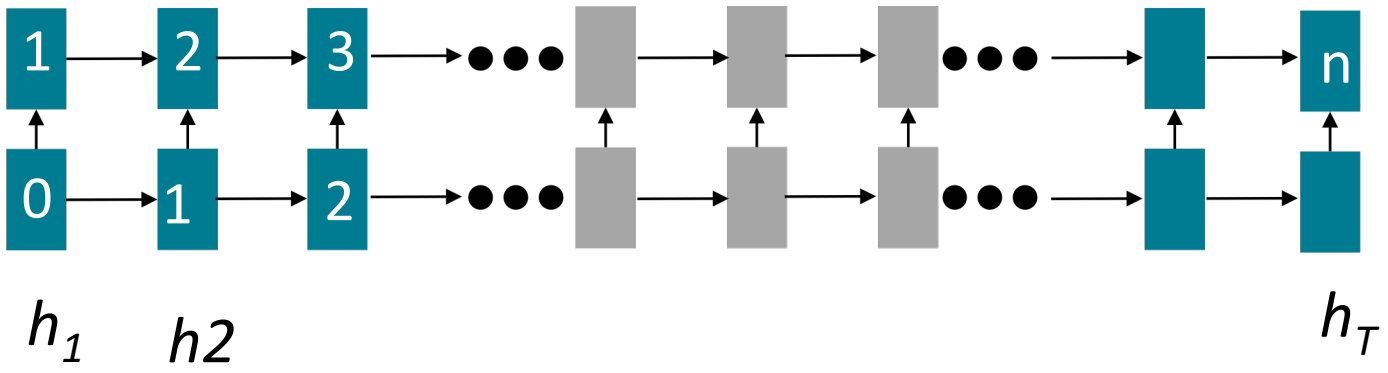
- **O(sequence length)** steps for distant word pairs to interact means:
  - Hard to learn long-distance dependencies (because gradient problems!)
  - Linear order of words is "baked in"; we already know linear order isn't the right way to think about sentences…



Info of ***chef*** has gone through O(sequence length) many layers!

Issues with recurrent models: Lack of parallelizability

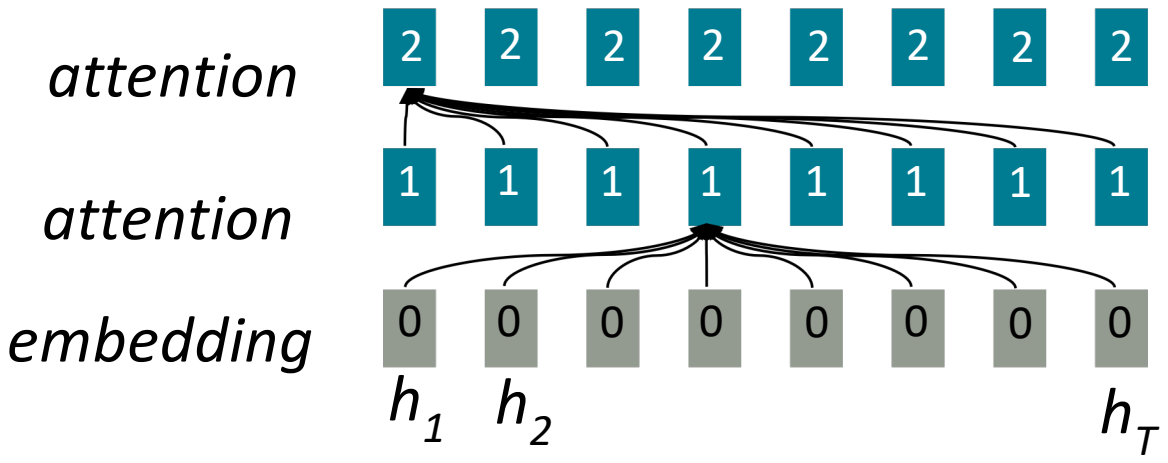
- Forward and backward passes have **O(sequence length)** unparallelizable operations
  - GPUs can perform a bunch of independent computations at once! But
  - future RNN hidden states can't be computed in full before past RNN hidden states have been computed
  - Inhibits training on very large datasets!



Numbers indicate min # of steps before a state can be computed

If not recurrence, then what? How about attention?

- **Attention** treats each word's representation as a **query**to access and incorporate information from **a set of values.**
  - We saw attention from the **decoder** to the **encoder**; today we'll think about attention **within a single sentence**.
- Number of unparallelizable operations does not increase with sequence length.
- Maximum interaction distance: O(1), since all words interact at every layer!

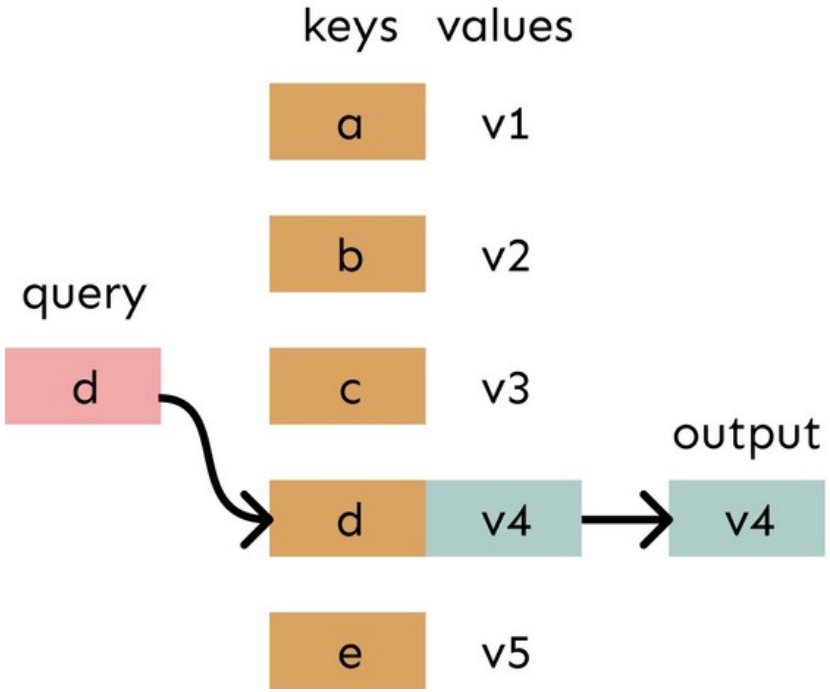


All words attend to all words in previous layer; most arrows here are omitted

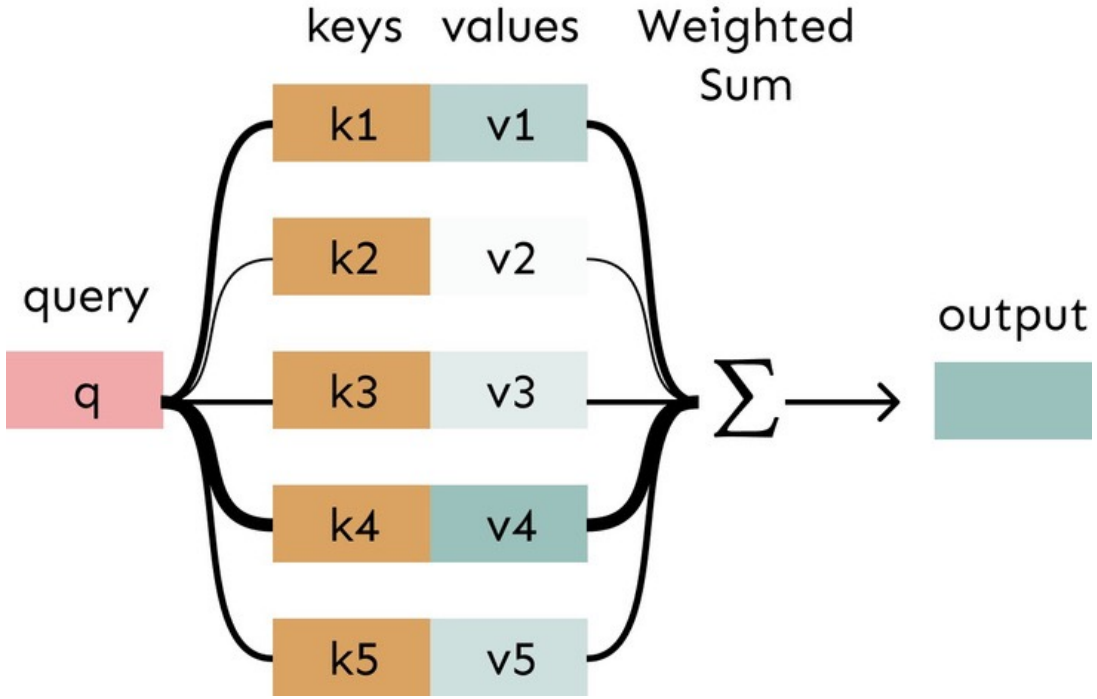
Attention as a soft, averaging lookup table

We can think of **attention**as performing fuzzy lookup in a key-value store.

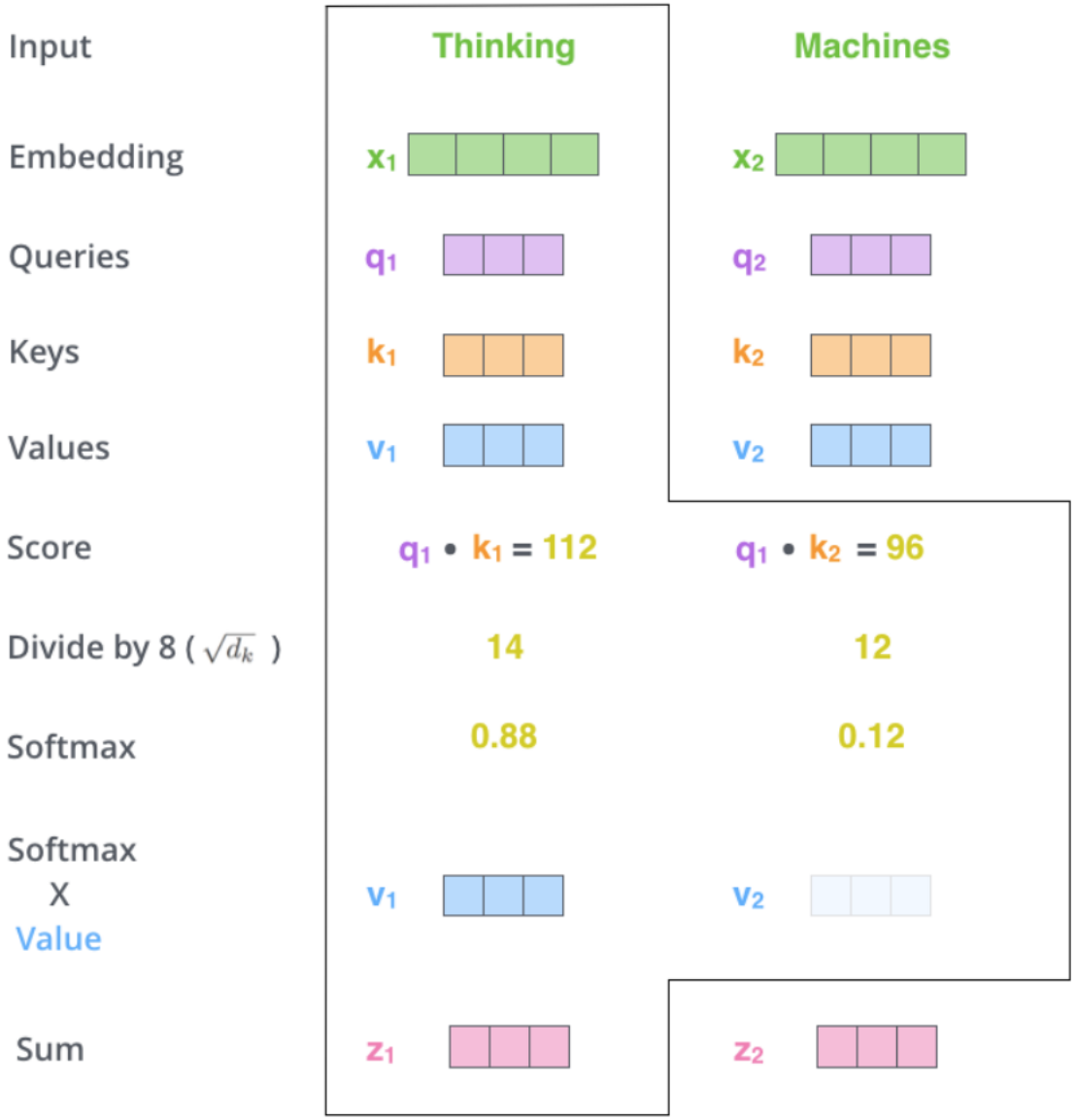
In a **lookup table**, we have a table of **keys** that map to **values**. The **query**matches one of the keys, returning its value.



In **attention**, the **query**matches all **keys***softly*, to a weight between 0 and 1. The keys' **values** are multiplied by the weights and summed.



Self-Attention in the Transformer Encoder



1. create three vectors from each of the encoder's input vectors (embedding of each word)

2. calculate a score by taking the dot product of the query vector with the key vector of the respective word we're scoring (thinking)

3 Divide the scores by 8 (the square root of the dimension of the key vectors used in the paper – 64

4. Multiply each value vector by the softmax score

5. Sum up the weighted value vectors.

This produces the output of the self-attention layer at this position (for the first word thinking).

Recipe for (Vectorized) Self-Attention in the Transformer Encoder

- Step 2: Calculate attention scores between **query and keys.**

$$E = QK^T$$

- Step 3: Take the softmax to normalize attention scores.

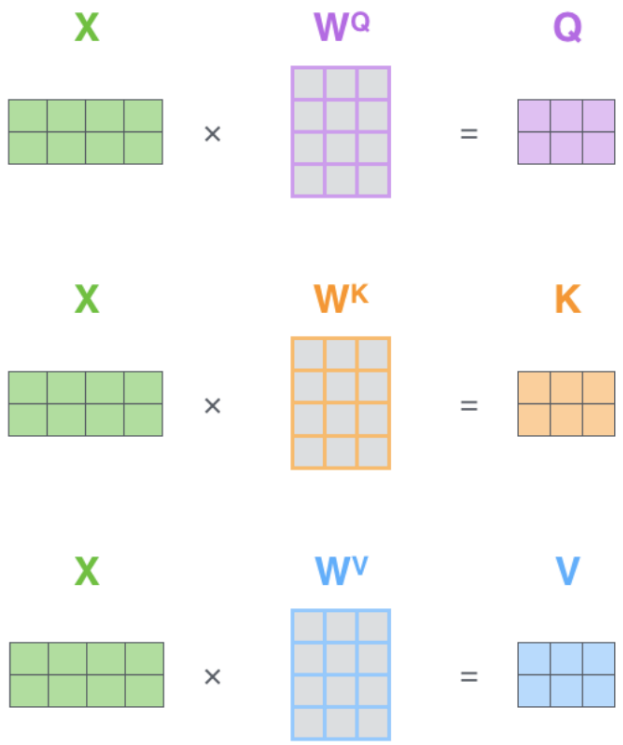
$$A = softmax(E)$$

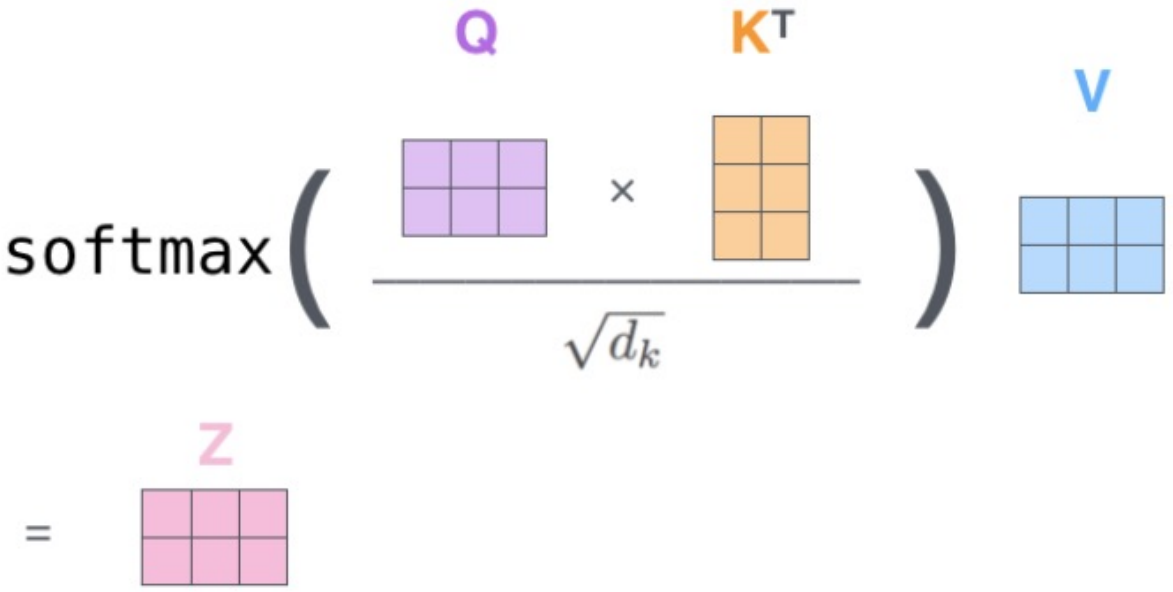
- Step 4: Take a weighted sum of **values**.

$$Output = AV$$

$$\boxed{Output = softmax\left(QK^T\right)V}$$

Matrix Calculation of Self-Attention





The self-attention calculation in matrix form

Calculate attention scores between **query** **and** **keys.**

Take the softmax to normalize attention scores.

Take a weighted sum of **values**.

$$E = QK^T$$

$$A = softmax(E)$$

$$Output = AV$$

$$Output = softmax(QK^T)V$$

Barriers and solutions for Self-Attention as a building block

Barriers

- Doesn’t have an inherent notion of order!

Solutions

Fixing the first self-attention problem: sequence order

Since self-attention doesn't build in order information, we need to encode the order of the sentence in our keys, queries, and values.

Consider representing each sequence index as a vector
$\boldsymbol{p}i \in \mathbb{R}d$, for $i \in \{1,2, \dots , n\}$ are position vectors

- Easy to incorporate this info into our self-attention block: just add the $\boldsymbol{p}i$ to our inputs!
- Recall that $\boldsymbol{x}i$ is the embedding of the word at index i. The positioned embedding is:

$$\boldsymbol{x}i = \boldsymbol{x}i + \boldsymbol{p}i$$

In deep self-attention networks, we do this at the first layer! You could concatenate them as well, but people mostly just add…

Position representation vectors through sinusoids

- It generates a unique vector for each position in the sequence, using a combination of sine and cosine functions.

- The encoding vector has the same dimensionality as the token embeddings, allowing them to be simply added together.

- Different dimensions in the encoding vector correspond to sinusoids of different frequencies, creating a spectrum from high to low frequencies.

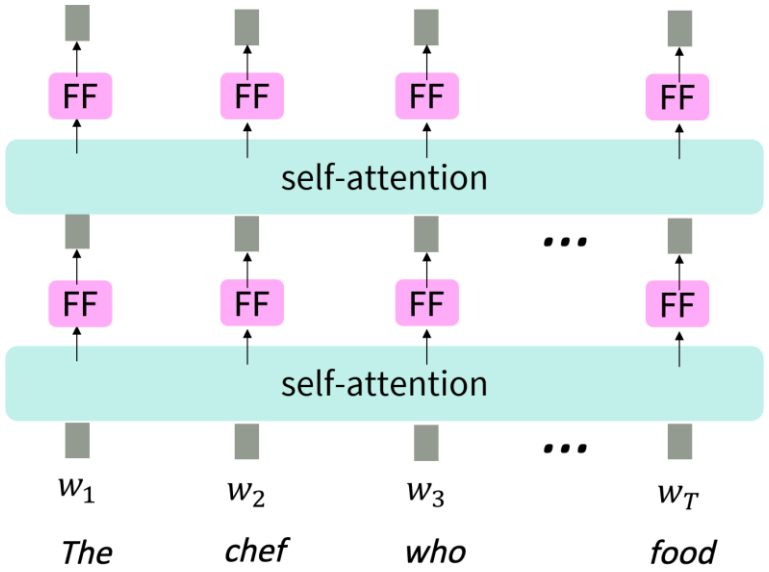
- This approach allows the model to easily attend to relative positions, as the encoding for any fixed offset can be represented as a linear function of the encoding at a given position.

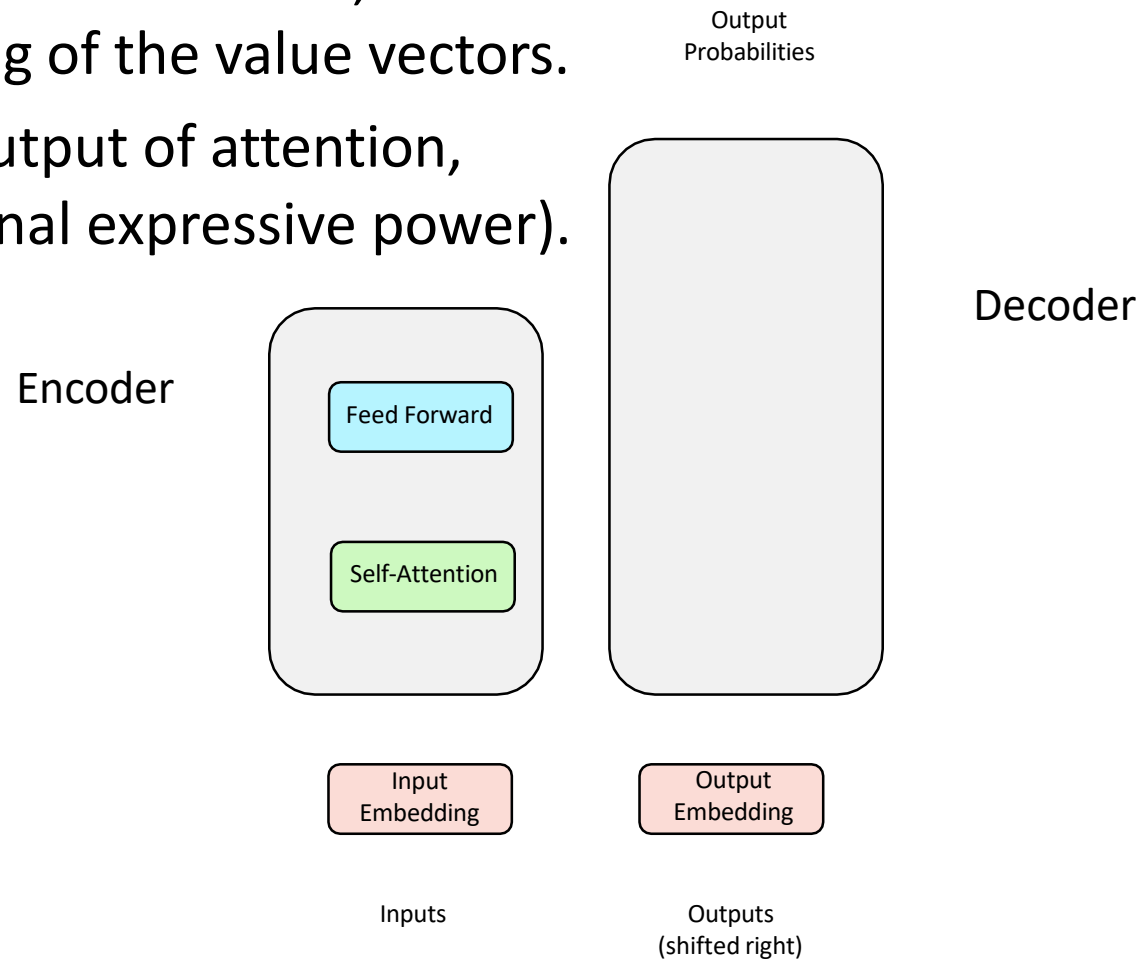
$$PE_{(pos,i)} = \begin{cases} \sin\left(\frac{pos}{10000^{i/d_{model}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{pos}{10000^{(i-1)/d_{model}}}\right) & \text{if } i \text{ is odd} \end{cases}$$

But attention isn't quite all you need!

- **Problem:** Since there are no element-wise non-linearities, self-attention is simply performing a re-averaging of the value vectors.
- **Easy fix:** Apply a feedforward layer to the output of attention, providing non-linear activation (and additional expressive power).

$$m_i = MLP(\text{output}_i)$$
$$= W_2 * \text{ReLU}(W_1 \times \text{output}_i + b_1) + b_2$$





Barriers and solutions for Self-Attention as a building block

Barriers	Solutions
• Doesn't have an inherent notion of order!	• Add position representations to the inputs
• No nonlinearities for deep learning magic! It's all just weighted averages	• Easy fix: apply the same feedforward network to each self-attention output.

• Need to ensure we don't "look at the future" when predicting a sequence
  • Like in machine translation
  • Or language modeling	

Masking the future in self-attention

- To use self-attention in **decoders**, we need to ensure we can't peek at the future.
- At every timestep, we could change the set of **keys and queries**to include only past words. (Inefficient!)
- To enable parallelization, we **mask out attention**to future words by setting attention scores to $-\infty$.

$$e_{ij} = \begin{cases} q_i^\top k_j, j \leq i \\ -\infty, j > i \end{cases}$$

We can look at these (not greyed out) words

For encoding these words

	[START]	The	chef	who
[START]		– ∞	– ∞	– ∞
The			– ∞	–
chef				∞ –
who				∞

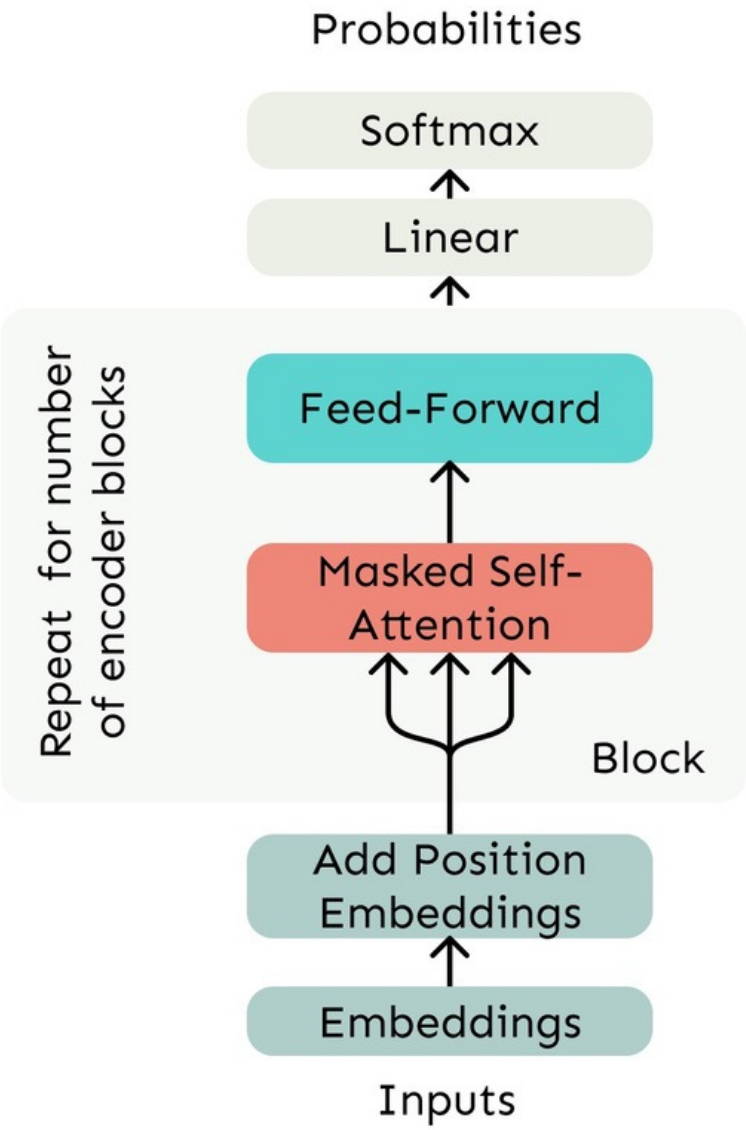
Barriers and solutions for Self-Attention as a building block

Barriers	Solutions
• Doesn't have an inherent notion of order!	• Add position representations to the inputs
• No nonlinearities for deep learning magic! It's all just weighted averages	• Easy fix: apply the same feedforward network to each self-attention output.

• Need to ensure we don't "look at the future" when predicting a sequence
  • Like in machine translation
  • Or language modeling
	• Mask out the future by artificially setting attention weights to 0!

Necessities for a self-attention building block:

- **Self-attention**:
  - the basis of the method.
- **Position representations**:
  - Specify the sequence order, since self-attention is an unordered function of its inputs.
- **Nonlinearities**:
  - At the output of the self-attention block
  - Frequently implemented as a simple feed-forward network.
- **Masking**:
  - In order to parallelize operations while not looking at the future.
  - Keeps information about the future from "leaking" to the past.



Next

- Transformers and BERT